Java Database & JDBC:

A database is a collection of organized information that can be stored, accessed, updated, and managed.

Example: A college database stores student details such as student ID, name, marks, and department.

JDBC (Java Database Connectivity) is a Java API that allows Java applications to connect to databases and perform operations such as inserting, retrieving, updating, and deleting data.

Learn database concepts, SQL fundamentals, and MySQL database setup:

1. Database Concepts

Definition: A database is an organized collection of data that can be easily stored, accessed, updated, and managed.

Example: A college database stores student details such as student ID, name, marks, and department.

Student ID | Name | Marks

----------|-------|------

101 | Ravi | 85

102 | Sita | 90

103 | Rani | 78

Benefits:

- Stores data in an organized way.
- Makes searching and updating data easy.
- Reduces unnecessary data duplication.

2. SQL Fundamentals

Definition: SQL (Structured Query Language) is a language used to create, retrieve, update, and delete data in a database.

Example: To display all student records from a students table:

SELECT * FROM students;

Other basic SQL commands:

Command	Purpose	Example
CREATE	Creates a table	CREATE TABLE students (id INT, name VARCHAR(50));
INSERT	Adds data	INSERT INTO students VALUES (1, 'Ravi');
SELECT	Retrieves data	SELECT * FROM students;
UPDATE	Modifies data	UPDATE students SET name = 'Raju' WHERE id = 1;
DELETE	Deletes data	DELETE FROM students WHERE id = 1;

3. MySQL Database Setup

Definition: MySQL database setup is the process of installing and configuring MySQL so that we can create databases, store data, and execute SQL queries.

Example: Create a database for a college using MySQL.

CREATE DATABASE college;

Select the database:

USE college;

Create a table:

```
CREATE TABLE students (
    id INT,
    name VARCHAR(50),
    marks INT
);
```

Insert a record:

```
INSERT INTO students VALUES (1, 'Ravi', 85);
```

Display the record:

```
SELECT * FROM students;
```

Output:

id	name	marks
1	Ravi	85

Note: These SQL commands can be executed using MySQL Workbench or the MySQL command-line client after MySQL is installed and running.

Quick Revision:

- **Database:** Stores and organizes data.
- **SQL:** Used to communicate with a database.
- **MySQL:** A database management system used to store and manage data.

Understand JDBC architecture, connections, statement execution, and prepared statements:

1. JDBC Architecture

Definition: JDBC (Java Database Connectivity) architecture explains how a Java application communicates with a database using the JDBC API and JDBC driver.

Example:

When a Java application retrieves student marks from a MySQL database, JDBC helps the application communicate with the database.

Working process:

Java Application → JDBC API → JDBC Driver → MySQL Database

Benefits:

- Connects Java applications to databases.
- Executes SQL queries.
- Retrieves and modifies database records.

---

2. JDBC Connection

Definition: A JDBC connection establishes communication between a Java application and a database.

Example:

```
import java.sql.*;

public class ConnectionExample {

   public static void main(String[] args) {

      try {

         Connection con = DriverManager.getConnection(

            "jdbc:mysql://localhost:3306/college",
```

```
            "root",
            "your_password"
        );
        System.out.println("Database connected!");
        con.close();
    } catch (SQLException e) {
        System.out.println(e.getMessage());
    }
  }
}
```

Output:

Database connected!

Explanation:

- DriverManager establishes the connection.
- getConnection() connects to the MySQL database.
- con.close() closes the connection.

Note: MySQL must be installed and running, and the database and credentials must be valid.

---

3. Statement Execution

Definition: Statement execution is the process of executing SQL commands in a database using a JDBC Statement object.

Example: Display all student records.

```
import java.sql.*;
public class StatementExample {
    public static void main(String[] args) {
        try {
            Connection con = DriverManager.getConnection(
                "jdbc:mysql://localhost:3306/college",
                "root",
                "your_password"
            );
            Statement st = con.createStatement();
            ResultSet rs = st.executeQuery(
                "SELECT * FROM students"
            );
            while (rs.next()) {
                System.out.println(rs.getString("name"));
            }
            rs.close();
            st.close();
            con.close();
        } catch (SQLException e) {
```

```
            System.out.println(e.getMessage());
        }
    }
}
```

Example output:

Ravi

Sita

Rani

Explanation:

- createStatement() creates a statement object.
- executeQuery() executes a query that retrieves data.
- ResultSet holds the retrieved records.
- rs.next() moves to the next record.

---

4. PreparedStatement

Definition: PreparedStatement is a JDBC interface used to execute parameterized SQL queries. It helps prevent SQL injection when parameters are bound correctly.

Example: Insert a student record into the database.

```
import java.sql.*;

public class PreparedExample {

    public static void main(String[] args) {

        try {
```

```
            Connection con = DriverManager.getConnection(
                "jdbc:mysql://localhost:3306/college",
                "root",
                "your_password"
            );
            PreparedStatement ps = con.prepareStatement(
                "INSERT INTO students VALUES (?, ?, ?)"
            );
            ps.setInt(1, 101);
            ps.setString(2, "Ravi");
            ps.setInt(3, 85);

            ps.executeUpdate();
            System.out.println("Student inserted!");
            ps.close();
            con.close();
        } catch (SQLException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

Output:

Student inserted!

Explanation:

- ? represents a parameter in the SQL query.
- setInt() supplies an integer value.
- setString() supplies a string value.
- executeUpdate() executes an INSERT, UPDATE, or DELETE statement.

Note: This example assumes the students table has three columns: id, name, and marks. Run the insert only once if you want to avoid duplicate student IDs.

Quick Revision Table

Topic	Simple Definition
JDBC Architecture	Explains how Java communicates with a database.
JDBC Connection	Establishes communication between Java and a database.
Statement Execution	Executes SQL commands using a Statement object.
PreparedStatement	Executes parameterized SQL queries safely and efficiently.

Important interview question: What is the difference between Statement and PreparedStatement?

Answer: Statement executes SQL queries directly, while PreparedStatement uses parameters (?) to supply values to a query. It is generally preferred for queries involving user input because it helps prevent SQL injection.

Perform database CRUD operations and transaction logic securely from a Java CLI application:

Java JDBC: CRUD Operations and Transaction Logic

1. CRUD Operations

Definition: CRUD stands for Create, Read, Update, and Delete. These are the four basic operations used to manage data in a database through a Java application.

Example

Suppose we have a students table in a college database.

1. Create (Insert Data)

Adds a new student record to the database.

INSERT INTO students VALUES (101, 'Ravi', 85);

2. Read (Retrieve Data)

Displays all student records.

SELECT * FROM students;

3. Update (Modify Data)

Changes the marks of student 101.

UPDATE students SET marks = 90 WHERE id = 101;

4. Delete (Remove Data)

Deletes student 101 from the database.

DELETE FROM students WHERE id = 101;

Benefits:

- Makes data management easy.
- Allows users to add and retrieve records.
- Helps modify and remove existing records.

---

2. Transaction Logic

Definition: A database transaction is a group of database operations treated as one logical unit. Either all the operations succeed and are committed, or the changes are rolled back if an error occurs.

Example: Suppose a bank transfers ₹500 from Account A to Account B.

1. Deduct ₹500 from Account A.
2. Add ₹500 to Account B.
3. If both operations succeed, commit the transaction.
4. If an operation fails, roll back the changes.

Java JDBC Example

```
import java.sql.*;

public class TransactionExample {

   public static void main(String[] args) {

      try (Connection con = DriverManager.getConnection(

            "jdbc:mysql://localhost:3306/bank",

            "root", "your_password")) {

         con.setAutoCommit(false);

         try (PreparedStatement ps1 = con.prepareStatement(
```

```
            "UPDATE accounts SET balance = balance - ? WHERE id =
?");
             PreparedStatement ps2 = con.prepareStatement(
            "UPDATE accounts SET balance = balance + ? WHERE id =
?")) {
            ps1.setDouble(1, 500);
            ps1.setInt(2, 1);
            int debit = ps1.executeUpdate();
            ps2.setDouble(1, 500);
            ps2.setInt(2, 2);
            int credit = ps2.executeUpdate();

            if (debit != 1 || credit != 1) {
               throw new SQLException("Account not found");
            }
            con.commit();
            System.out.println("Transfer successful!");

        } catch (SQLException e) {
            con.rollback();
            System.out.println("Transfer failed. Changes rolled back.");
        }
```

```
        } catch (SQLException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

Example output:

Transfer successful!

Explanation:

- setAutoCommit(false) allows manual transaction control.
- commit() saves the transaction's changes.
- rollback() undoes uncommitted changes if an operation fails.
- PreparedStatement helps prevent SQL injection.

Note: This example assumes the accounts table contains id and balance columns, and both accounts exist. For a real banking application, also validate sufficient funds and use appropriate database constraints and locking.

---

3. Secure Database Operations from a Java CLI Application

Definition: A Java CLI (Command-Line Interface) application allows users to manage database records by entering commands or choices in a terminal. Secure database operations use techniques such as PreparedStatement to protect against SQL injection.

Example:

===== Student Management =====

1. Add Student

2. View Students

3. Update Student

4. Delete Student

5. Exit

Enter your choice: 1

Enter Student ID: 101

Enter Name: Ravi

Enter Marks: 85

Student added successfully!

The Java program accepts the user's choice, executes the corresponding JDBC operation, and displays the result.

Quick Revision

Topic	Simple Definition
CRUD Operations	Create, Read, Update, and Delete database records.
Transaction	A group of operations treated as one logical unit.
Commit	Saves transaction changes.
Rollback	Reverses uncommitted transaction changes.
Java CLI	A program operated through a command-line terminal.
Secure JDBC	Uses parameterized queries and appropriate transaction controls to protect database operations.

Build a Java + MySQL Management System linking Java frontend to real-world database tables:

1. Java + MySQL Management System

Definition: A Java + MySQL Management System is an application that uses Java to interact with a MySQL database to store, retrieve, update, and delete information.

Example: A Student Management System allows users to add student details, view student records, update marks, and delete records using a Java application connected to MySQL.

2. How Does It Work?

The system works in three steps:

1. **Java Frontend:** Accepts user input and displays information.
2. **JDBC:** Connects the Java application to MySQL.
3. **MySQL Database:** Stores and manages student records.

Working flow:

Java Application → JDBC → MySQL Database → Results displayed in Java

3. Example: Student Management System

MySQL Database Table

First, create a database and table:

```
CREATE DATABASE college;

USE college;

CREATE TABLE students (

   id INT PRIMARY KEY,
```

```
    name VARCHAR(50),
    marks INT
);
```

Java Program

This simple program inserts a student record into MySQL.

```
import java.sql.*;
public class StudentManagement {
    public static void main(String[] args) {
        try {
            Connection con = DriverManager.getConnection(
                "jdbc:mysql://localhost:3306/college",
                "root", "your_password"
            );
            PreparedStatement ps = con.prepareStatement(
                "INSERT INTO students VALUES (?, ?, ?)"
            );
            ps.setInt(1, 101);
            ps.setString(2, "Ravi");
            ps.setInt(3, 85);
            ps.executeUpdate();
```

```
            System.out.println("Student added successfully!");
            ps.close();
            con.close();
        } catch (SQLException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

Output:

Student added successfully!

Database result:

ID	Name	Marks
101	Ravi	85

4. Benefits

- Stores data permanently in a database.
- Makes data retrieval and updating easy.
- Connects Java applications with real-world databases.
- Supports CRUD operations.
- Helps develop applications such as student, employee, library, and inventory management systems.

Quick Revision

Java + MySQL Management System: A database application in which Java handles user interaction, JDBC connects Java to MySQL, and MySQL stores the data.

Real-world example: A Library Management System that stores book details, manages student borrowing records, and tracks book returns using Java and MySQL.